

Phase 4 – Project Report
CMPT 276 – Intro to Software Engineering
Escape from the Burnaby Mountain Prison

Group Members:

Jagdeep Sidhu
Arsh Behniwal
Taranjot Aujla
Dominic Janus
Nick Zhou

April 7, 2026
Simon Fraser University

Contents

1	The Game	3
1.1	Game Overview	3
1.2	Original Plan and Design	3
1.3	Final Game	4
1.4	Changes and Lessons Learned	4
2	Tutorial	5

1 The Game

1.1 Game Overview

Escape from the Burnaby Mountain Prison is a 2D arcade styled maze game, where the player takes on the role of an inmate who has been charged with multiple counts of using AI on assignments. The goal of the inmate is to escape the Burnaby Mountain Prison before the summer ends. The game opens up with a story sequence which sets the scene before the player is dropped into the prison, the game features three selectable difficulty levels that adjust the number of starting lives and the number of guards patrolling the map.

The player is forced to navigate through a grid-based prison map by using standard directional controls, moving one tile at a time through corridors, open areas, and cell blocks, while avoiding the traps and guards scattered all throughout the map. The map is a large scrolling environment with a camera that follows the player, and contains a fog of war effect that limits visibility to a radius around the player, this creates a sort of tension as guards and traps can appear from out of nowhere.

To win, the player must collect all three rewards scattered throughout the map - a laptop, a student ID, and a pet raccoon; once all these things have been collected, they must then reach the designated exit tile. There are also gold coins scattered along the way which count as bonus score pickups and the player loses if either their lives are depleted or their score drops below zero.

In order to make the game more challenging, hazards such as handcuffs, spoiled milk, parking tickets, and a bear are scattered throughout the map and apply negative effects to the player upon contact. These effects include slower movement, inverting the controls, and reducing the player's current score. In order to counteract all these debuffs, there are also powerups such as a coffee cup which grants the player a speed boost, a snowflake which temporarily freezes all the guards, and a doctor's note which provides a brief moment of invulnerability.

1.2 Original Plan and Design

Our original plan for Escape from the Burnaby Mountain Prison was to develop a 2D arcade maze game. The game was planned out to be controlled by a central Game class, managing the main loop, timing, and game states, including menu, playing, level complete, and game over.

The prison was designed as a grid map with walls, barriers, a start point, and an exit. Both the inmate and guards were planned to share a common Entity base, aiming to reuse movement and collision logic across the characters. We also planned several map items, including rewards, coins, hazards, and powerups, to make exploration more engaging.

Our original design focused on guard behaviour and kept a simple arcade-style interface. Guards were planned to patrol and respond to the player using different states, while the mock UI focused on keeping the map simple and displaying only key information such as time and score.

1.3 Final Game

The final version of Escape from the Burnaby Mountain Prison successfully delivers on our core fundamentals from our initial planning stages but introduces a much deeper layer of strategy and tension while refining game mechanics further. Rather than a simple, fully visible maze, the final version incorporates a fog effect that restricts the player’s vision, forcing them to carefully explore the different areas and rely on their memory to navigate towards the exit.

The item economy also expanded significantly beyond basic score-boosting collectibles. We implemented a shared MapItem system that fills the map with specific strategic elements. Players can utilize three unique powerups: Renaissance Cofee (grants a 2x speed boost), Snowflakes (temporarily freezes all guards), and Doctor’s Notes (restores a lost life). To balance this, players must also actively avoid four environmental hazards: Handcuffs (prevents item collection), Parking Tickets (reduces movement speed), Spoiled Milk (inverts player controls) and Bear attacks (instantly removes one life from the player). Whenever an item is triggered, a custom UI popup message is shown to give the player immediate feedback.

The most significant change we made from our original design would be the guard AI. Instead of moving randomly or using a basic directional tracking algorithm, the final guards operate using a Breadth-First Search (BFS) pathfinding algorithm. They actively transition between an orange Patrol state and a red Chase state when the player enters their line of sight. These systems are all tied together by a stable 30-FPS game loop, ensuring smooth, consistent gameplay as the player collects their final required rewards, like the SFU Raccoon or Student ID, and makes a final dash for the exit.

1.4 Changes and Lessons Learned

Throughout the development of our game, we made several key changes to our original design due to practical constraints, technical challenges, and an increase in understanding of the implementation/ system as our work progressed.

We were hit with a technical challenge while implementing our initial design for guard movement, which relied on simple direction tracking. During testing and gameplay, we found this approach caused guards to get stuck on walls and did not feel realistic. To fix the issue, we changed the logic to a breadth-first search pathfinding algorithm approach. While this newer implementation was more complex, it improved the gameplay quality and realism. However, running the pathfinding calculations every frame introduced lag to the guard movement. Taking into consideration the performance, we optimized the program by caching paths and recalculating them periodically.

Additionally, we also made changes based on performance and gameplay feel. We reduced the game loop from our intended 60FPS to 30FPS after discovering the higher frame rate made the game feel unstable and did not match the arcade-like gameplay we were aiming for. This trade-off resulted in smoother and more consistent gameplay. We also made structural changes to the class layout to improve code maintainability and reduce duplication. Initially, we had separate implementations for rewards, powerups, and hazards; this led to duplicated code and collection logic. After developing and refactoring our code, we decided to create a

shared MapItem class that would allow us to use shared functionality throughout multiple classes. This change made the code much cleaner and easier to add on to. Additionally, there were also many features such as scoring mechanics, difficulty increases, and UI improvements, which were not fully defined in our initial design. These features evolved as we continued to develop and test the game as we discovered better design ideas that would enhance gameplay and user experience.

One of the most important lessons we learned was the value of good software design. Applying concepts such as inheritance, like in the Entity and MapItem classes, significantly reduced code duplication and made it easier to extend from previous work. We also learned the importance of breaking down larger methods into multiple smaller methods focused on specific components to make the code easier to understand and maintain. This project also developed debugging and problem-solving skills. The many bugs we found while developing the game, such as guard pathfinding, collision edge cases, and audio bugs, required extensive testing and research. Learning how to isolate problems and test created solutions was crucial to completing the game. From a teamwork perspective, we learned the importance of clear communication and role distribution. Using a structured workflow and clearly defining responsibilities allowed all team members to work in parallel without running into merge conflicts. Regular meetings and continuous communication allowed us to quickly resolve issues and stay on track to meet specified deadlines. Finally, we learned firsthand the benefits of working in an agile development process. Many of our initial ideas were not optimal, and it was only through testing and continued development that we were able to come up with effective solutions. Being flexible and willing to adapt our original design was key to the project's success.

2 Tutorial

The game uses standard movement controls, WASD and the arrow keys to navigate the map. Items and hazards are automatically collected/ applied when the player collides with them. The main objective of the game is to collect all three rewards scattered across the map while avoiding guards and hazards. Once collected, the player can navigate towards the exit and complete the game. Players can choose the game difficulty; increased difficulty results in fewer lives and more guards. When a player loses all lives or goes below a score of 0, the game ends.

Refer to the video demo for more information: <https://youtu.be/UdoQQZyG0StY?si=w-JmeqDFnT5LDI3E>